Deep Generative Models Part 3

Flow Matching, Controlling generation, and Parameter Efficient Transfer Learning

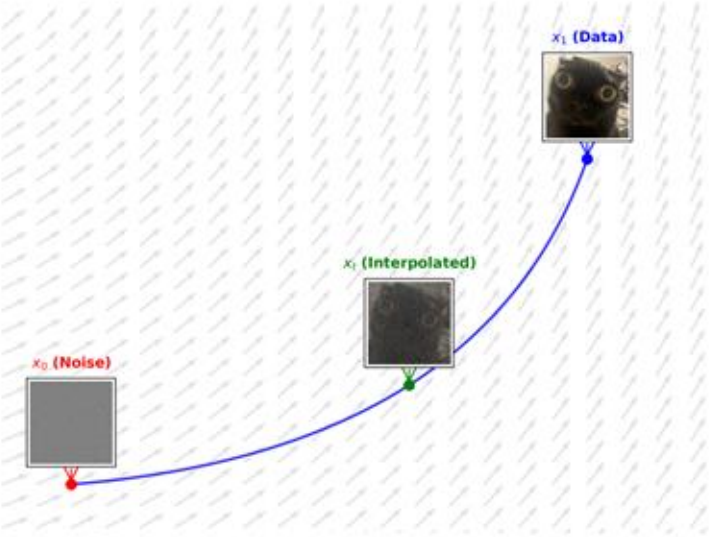
Lecture Plan

1. Flow matching
    a. ODE
    b. Vector field and score function
    c. Flow Matching Loss and Conditional Flow Matching Loss
    d. Conditional/Marginal vector field
    e. Optimal Transportation
2. Controlling the generation process
    a. Condition on a Prompt
    b. Classifier free guidance
3. Parameter Efficient Fine-Tuning
    a. LoRA

Flow matching

Notation

- time $t \in [0, 1]$
- x_0 = initial noise, x_t = interpolated data at t, x_1 = original data
- z = x_1 = a data sample
- $p_t(x)$ = probability path (time-dependent pdf), $\int p_t(x)\mathrm{d}x = 1$
- $v_t(x), u_t(x)$ = (time-dependent) velocity/vector field
- $q(x), p_{\text{data}}(x)$ = data distribution



Back to ODE

A path if we start at $X_0 = x_0$ at $t = 0$

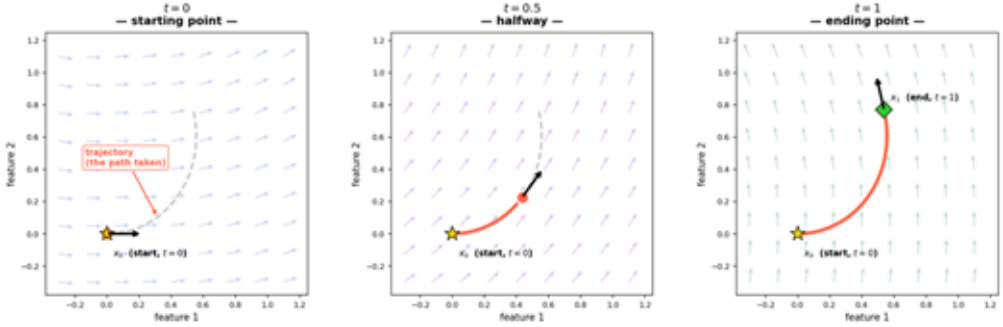
ODE

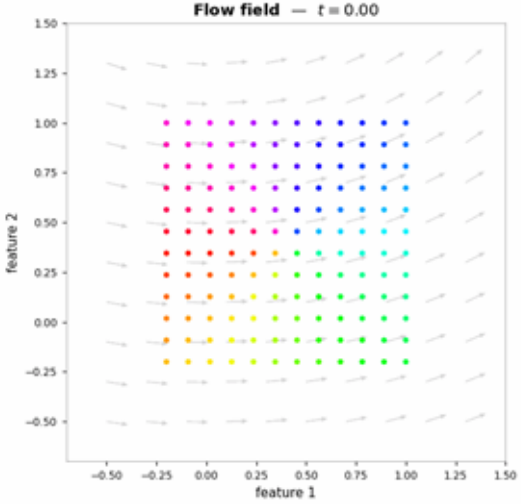
$$\frac{\mathrm{d}}{\mathrm{d}t} X_t = u_t(X_t)$$
$$X_0 = x_0$$

Flow ODE

$$\frac{\mathrm{d}}{\mathrm{d}t} \psi_t(x_0) = u_t(\psi_t(x_0))$$
$$\psi_0(x_0) = x_0$$

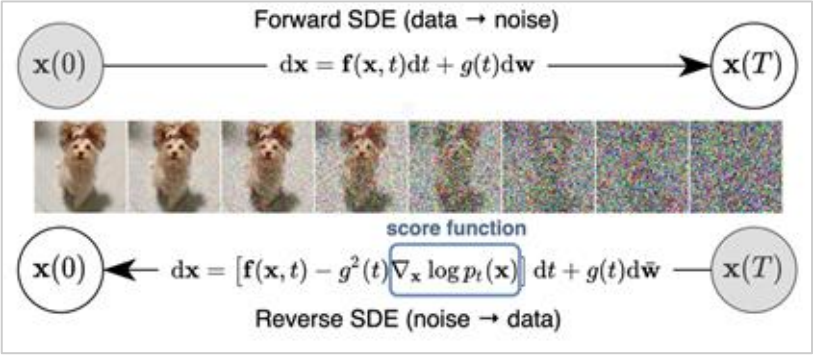
- Ordinary differential equations (ODEs)
  - Trajectory (solution): $X : [0,1] \to \mathbb{R}^d, \ t \mapsto X_t$
  - Vector field (velocity in space): $u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \ (x,t) \mapsto u_t(x)$
  - Flow (specific instantiation if we start at x0 at t=0): $\psi : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \ (x_0,t) \mapsto \psi_t(x_0)$

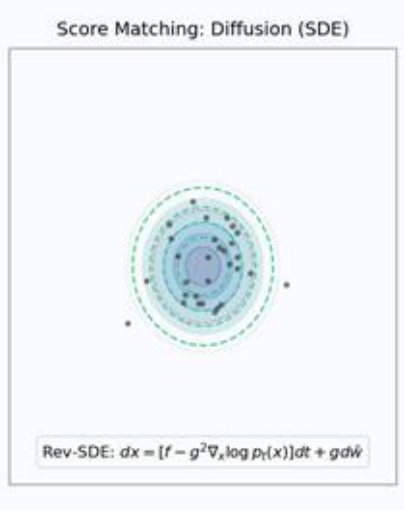




Revisit: Score matching Diffusion (SDE)

Score-Based Diffusion uses SDEs to reverse the noise and generate data.





The generative paths are highly stochastic and noisy

- The score matching paths are so complex and noisy, you need hundreds or thousands of tiny mathematical steps to solve the reverse SDE, which requires a high computational cost.

- **Indirect Objective**: We are learning the gradient of the log probability density (the score $\nabla_{\mathbf{x}} \log p_t$) which is an indirect way to model the actual vector field moving the particles.

Revisit: Continuous Normalizing Flow (CNF)

A CNF is used to transform pure noise (p_0) from some initial distribution to a target data distribution (p_1), defined by the ODE:

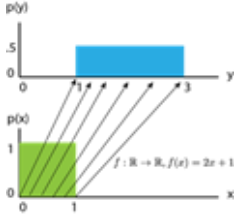
$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(x_0) = v_t^\theta(\psi_t(x_0))$$
$$\psi_0(x_0) = x_0$$

and transforms the probability path using

$$p_t = p_0(\psi_t^{-1}(x)) \left| \det \left[\frac{\partial \psi_t^{-1}}{\partial x}(x) \right] \right|$$

norm

$$\int p_t(x)\mathrm{d}x = 1$$



CNFs are deterministic, but they are computationally expensive due to the high number of ODE simulations required (the network must sequentially evaluate the vector field v at many time steps t along the trajectory).

https://arxiv.org/abs/2210.02747

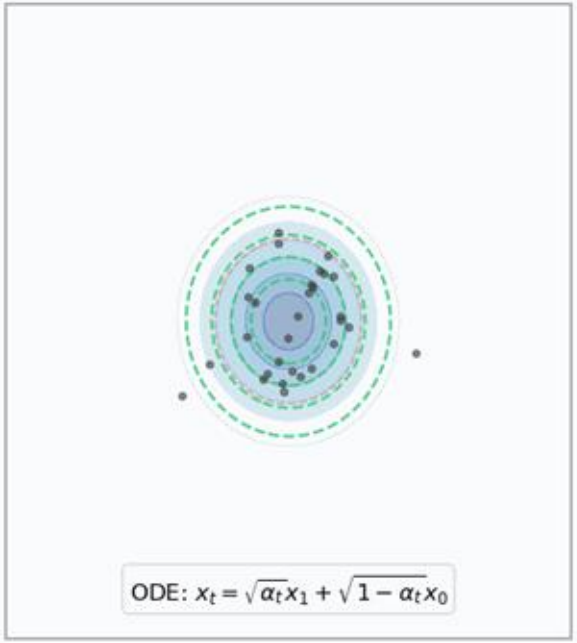
Flow matching

Flow matching is a process that tries to approximate an unknown data distribution p_{data} (we don't know the pdf, but we can sample from it via our dataset) by transforming an initial distribution p_{init}, e.g., N(0, I)

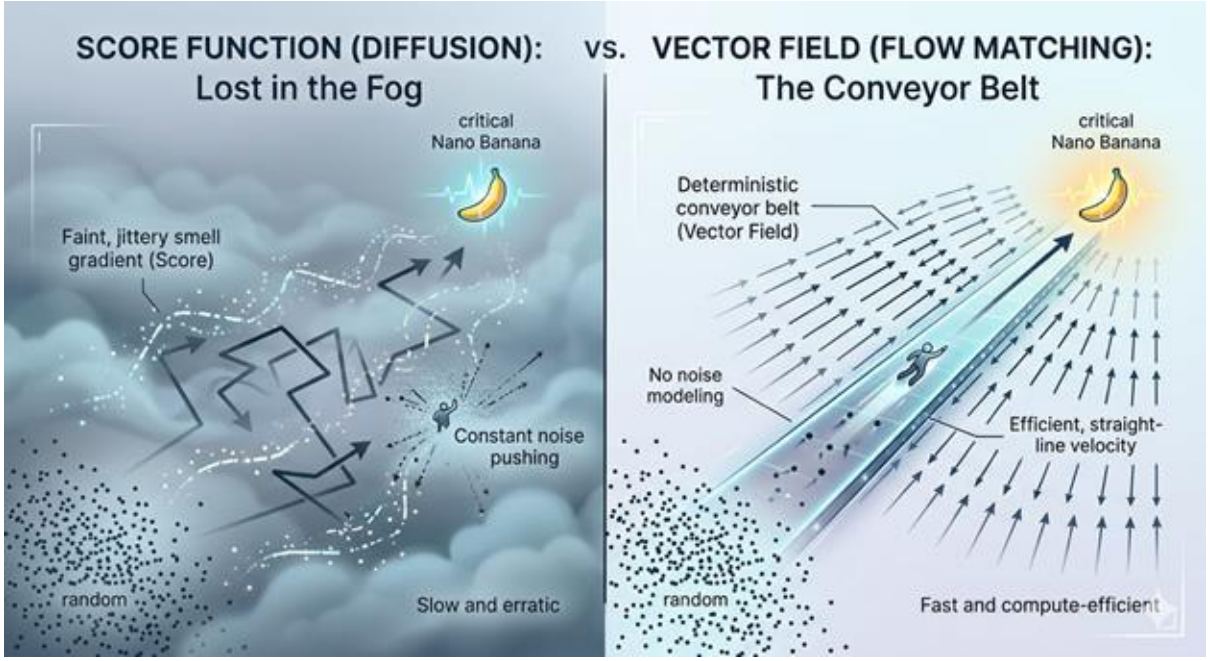
$$p_1(x) \approx p_{\text{data}}(x), \quad \text{where } p_0 = p_{\text{init}}$$

Goal: The model learns a vector field $v_t^\theta(x)$ to define a probability path that flows from p_0 to p_1

Example: Mapping the initial distribution p_0=N(0,I) (red dashed circle) to the unknown data distribution p_{data} (green dashed shape)



Score function vs Vector field



Training Flow Matching — Flow Matching Loss

Flow matching is trained on a MSE between a predicted vector field v_t^θ and a target vector field u_t

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\ x_t\sim p_t(x_t),} \left\| v_t^\theta(x_t) - \underbrace{u_t(x_t)}_{\text{intractable}} \right\|^2$$

Unfortunately, we face the same problem as in our last lecture: the closed-form of the target vector field, which is also used to generate the probability path, is intractable.
However, both can be addressed using the same approach: a **conditional** vector field (per sample).

$$z \sim p_{\mathrm{data}} \qquad u_t(x|x_1 = z) \quad \text{and} \quad p_t(x|x_1 = z)$$

random sample from the dataset

conditional vector field for a single sample

conditional probability path for a single sample

Training Flow Matching — Conditional VF

See Theorem 1 in the original paper for more details

The marginal probability paths and vector fields can be constructed by

$$p_t(x) = \int p_t(x|x_1 = z) p_{\text{data}}(z) \mathrm{d}z$$

$$p_t(x_1 = z|x)$$

Given the current noisy point x that we are looking at, what is the probability that it originally started out as the specific image z

Bayes' theorem

$$u_t(x) = \int u_t(x|x_1 = z) \underline{\frac{p_t(x|x_1 = z) p_{\text{data}}(z)}{p_t(x)}} \mathrm{d}z$$

(weigted) average across each sample

These equations show that the marginal vector field u_t generates the marginal probability path p_t (u_t,p_t satisfy *continuity equation*).

$$x_0 \sim p_{\text{init}}, \quad \frac{\mathrm{d}}{\mathrm{d}t} x_t = u_t(x_t) \quad \Rightarrow \quad x_t \sim p_t$$

Training Flow Matching — Conditional Flow Matching Loss

From the theorem, a marginal vector field can be trained by randomly sampling an individual data point and training on a specific vector field for that data point.

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{\substack{t\sim\mathcal{U}[0,1], z\sim p_{\mathrm{data}}, \\ x_t\sim p_t(x_t|x_1=z)}} \left\| v_t^\theta(x_t) - u_t(x_t|x_1 = z) \right\|^2$$

An additional theorem (Theorem 2) shows that the marginal flow matching loss is equal to the conditional flow matching loss plus some constant that is independent of the parameters θ, i.e.,

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathcal{L}_{\mathrm{CFM}}(\theta) + C$$

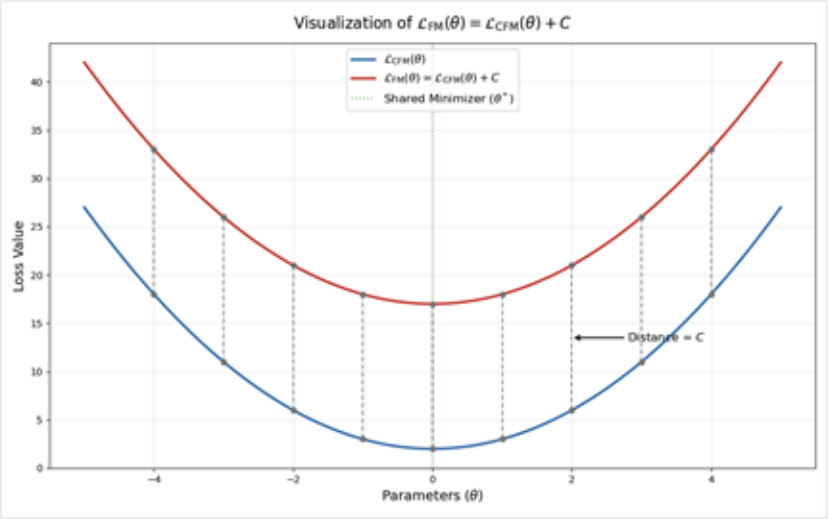
proof in the appendix.

Training Flow Matching — Conditional Flow Matching Loss

Observation: A marginal flow matching loss and the conditional flow matching loss share the same gradient and minimizer, so we can compute the intractable marginal flow matching loss through the tractable conditional flow matching loss.

$$\nabla_\theta \mathcal{L}_{\mathrm{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\mathrm{CFM}}(\theta)$$

$$\theta^* = \arg\min_\theta \mathcal{L}_{\mathrm{FM}}(\theta) = \arg\min_\theta \mathcal{L}_{\mathrm{CFM}}(\theta)$$

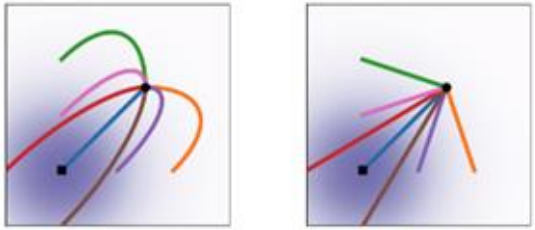


Choosing Path

After establishing the loss function, the next critical step is selecting the path that flows the initial noise distribution to the target data distribution.

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1], z\sim p_{\mathrm{data}}, \boxed{x_t\sim p_t(x_t|x_1=z)}} \left\| v_t^\theta(x_t) - u_t(x_t|x_1 = z) \right\|^2$$

We will define the structure of p_t. Recall that the marginal vector field u_t generates the marginal probability path p_t. Consequently, the path we select directly determines the complexity and shape of the resulting vector field u_t.



https://arxiv.org/abs/2210.02747

Definition: Gaussian path

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t; \alpha_{1-t} z, (1 - \alpha_{1-t}^2)\mathbf{I}_d)$$

when t=0: $\mu_0(x_1) = 0, \sigma_0(x_1) = 1$ (standard gaussian)

when t=1: $\mu_1(x_1) = x_1, \sigma_1(x_1) = \sigma_{\min} \approx 0$ (centered at data point x_1)

- Flow map: $\psi_t(x_0) = \sigma_t(z)x_0 + \mu_t(z)$ → Interpolation between x_0 and x_1

d/dt

- Vector field: $u_t(x_t|x_1) = \sigma_t'(x_1)\underbrace{\boxed{\frac{x_t - \mu_t(x_1)}{\sigma_t(x_1)}}}_{x_0} + \mu_t'(x_1)$

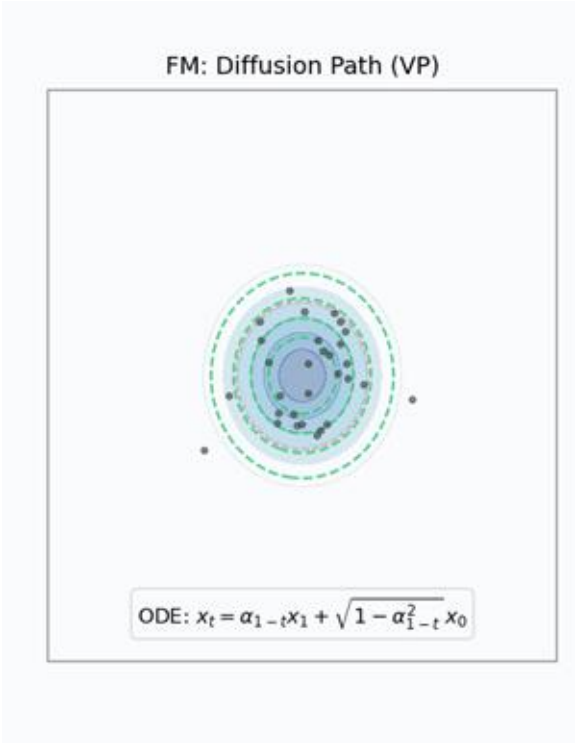
1. Diffusion Probability Path (Variance Preseving)

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t; \alpha_{1-t} z, (1 - \alpha_{1-t}^2)\mathbf{I}_d)$$

$$\text{where} \quad \alpha_t = e^{-\frac{1}{2}T(t)}$$

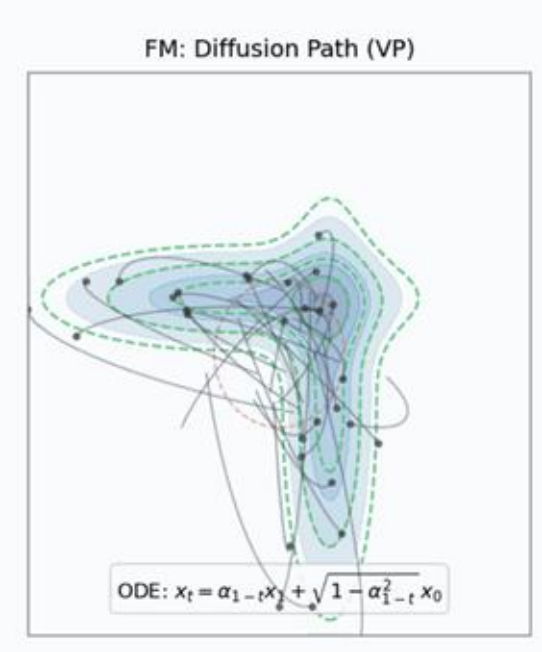
$$T(t) = \int_0^t \beta(s)\mathrm{d}s$$

$$\Rightarrow u_t(x_t|x_1 = z) = \frac{\alpha'_{1-t}}{1 - \alpha_{1-t}^2}(\alpha_{1-t} x_t - z)$$



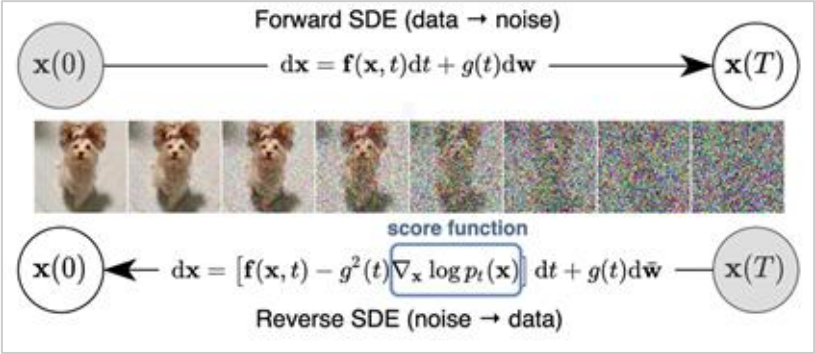
1. Diffusion Probability Path (VP) — “Overshoot”

The diffusion path suffers from an **"overshoot"** problem, causing the trajectory to backtrack unnecessarily.



1. Diffusion Probability Path (VP) — “Pure Noise”

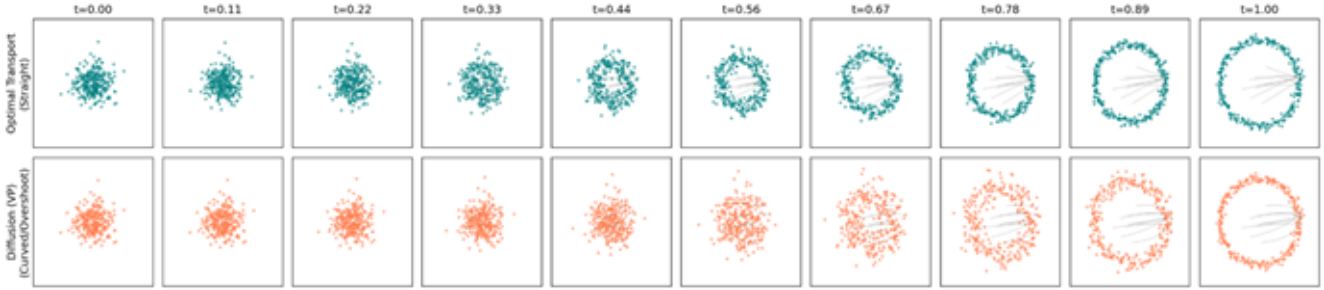
Diffusion models work by taking an image and gradually adding noise to it through the forward process. Mathematically, it requires an infinite number of time steps to completely turn the image into pure Gaussian noise.



However, because the training process must occur in finite time, the forward process used in the diffusion model only results in a close approximation of pure noise. In contrast, during the generation (sampling) process, the model starts denoising from true Gaussian noise, which causes a slight mathematical mismatch.

2. Optimal Transport probability path

Optimal Transport (OT) is the mathematical problem of finding the most efficient way to transform one probability distribution into another.



OT minimizes the transport cost. Particles under the OT displacement map are forced to move in straight line trajectories with constant speed. This avoids the curved, "overshooting," and backtracking paths typical of standard diffusion models.

Recall from WGANs: Earth Mover's (Wasserstein) Distance calculates the minimum cost to move mass. Here, Optimal Transport gives us the actual straight-line paths that achieve that minimum cost.

Note: Wasserstein-p computes distances based on Lp norm. WGAN use p = 1 which is more robust to outliers. p=2 is often used for Gaussians due to its closed-form solution.

2. Optimal Transport probability path

Note: An optimal conditional vector field does not guarantee an optimal marginal vector field!

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t; tz, (1-t)^2 \mathbf{I}_d)$$

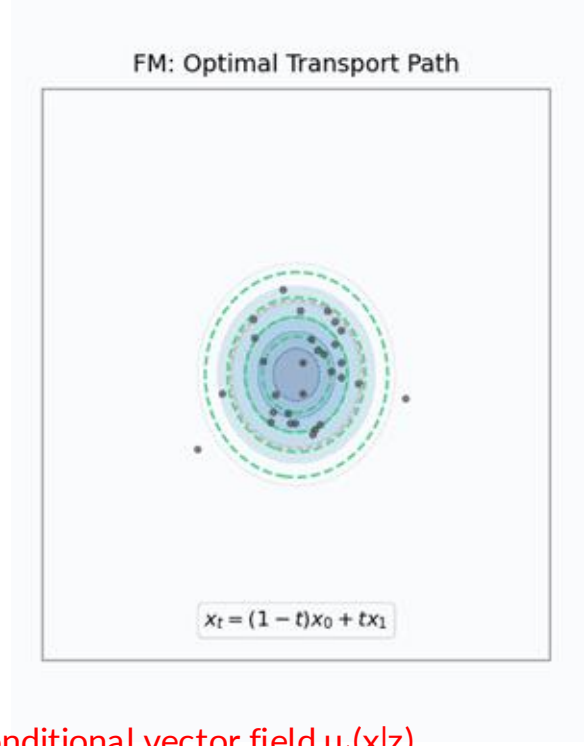
(This path is based on the "Wasserstein-2 optimal transport solution")

Mean and standard deviation change linearly over time.

$$u_t(x_t|x_1 = z) = \frac{z - x_t}{1 - t}$$

Then the conditional flow matching loss will be

$$\begin{aligned}\mathcal{L}_{\text{CFM}}(\theta) &= \mathbb{E}_{t\sim\mathcal{U}[0,1], z\sim p_{\text{data}}, x_t\sim p_t(x_t|x_1=z)} \left\| v_t^\theta(x_t) - \frac{z - x_t}{1-t} \right\|^2 \\ &= \mathbb{E}_{t\sim\mathcal{U}[0,1], z\sim p_{\text{data}}, x_0\sim\mathcal{N}(0,\mathbf{I}_d)} \left\| v_t^\theta(\underline{tz + (1-t)x_0}) - \underline{(z - x_0)} \right\|^2\end{aligned}$$



Trajectories follow straight paths with constant speed

linear interpolation!

"Rectified Flow"

conditional vector field $u_t(x|z)$

Training Flow Matching Model with OT path

Algorithm 3 Flow Matching Training Procedure (for Gaussian CondOT path $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$)

Require: A dataset of samples $z \sim p_{\text{data}}$, neural network u_t^θ

for each mini-batch of data **do**
  Sample a data example z from the dataset.
  Sample a random time $t \sim \text{Unif}_{[0,1]}$.
  Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
  Set

$$x = tz + (1-t)\epsilon \qquad \text{(General case: } x \sim p_t(\cdot \mid z))$$

  Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2 \qquad \text{(General case: } = \|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2)$$

  Update $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$.
end for

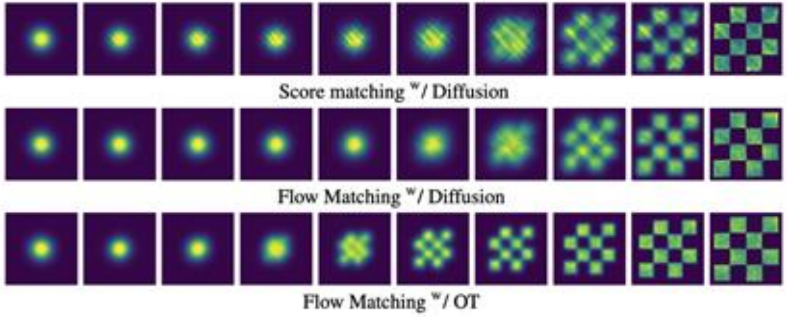
Algorithm 1 Sampling from a Flow Model with Euler method

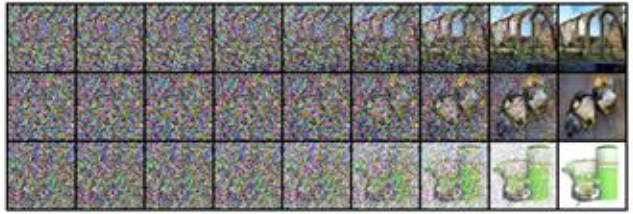
Require: Neural network vector field u_t^θ, number of steps n

Set $t = 0$
Set step size $h = \frac{1}{n}$
Draw a sample $X_0 \sim p_{\text{init}}$
for $i = 1, \ldots, n$ **do**
  $X_{t+h} = X_t + h u_t^\theta(X_t)$
  Update $t \leftarrow t + h$
end for
return X_1

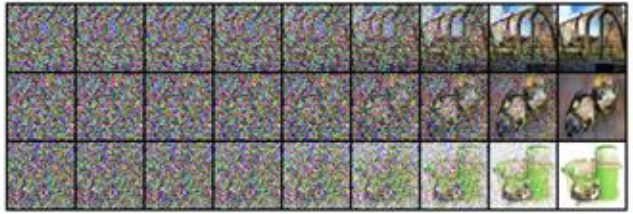
https://diffusion.csail.mit.edu/2026/docs/lecture_notes.pdf

2. Optimal Transport probability path

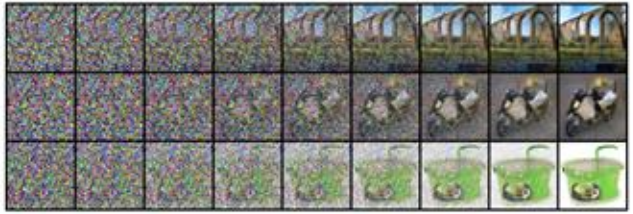




Score Matching w/ Diffusion



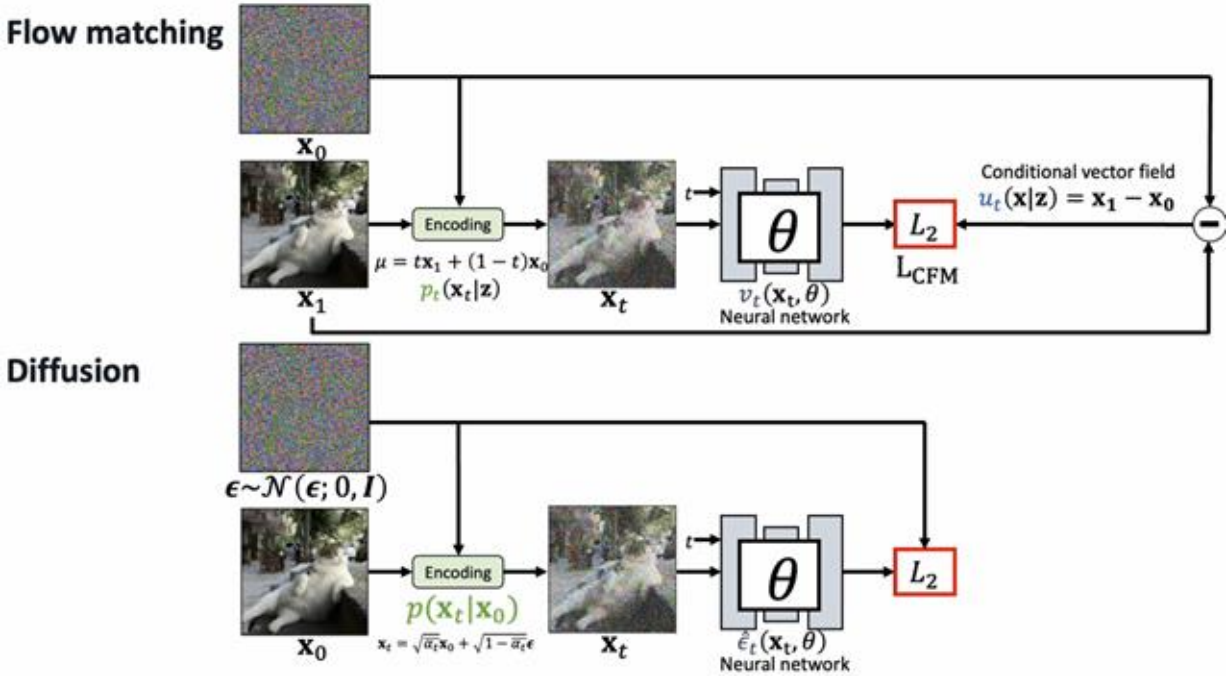
Flow Matching w/ Diffusion



Flow Matching w/ OT

Flow Matching for Generative Modeling: https://arxiv.org/abs/2210.02747

Flow and Diffusion model



https://youtu.be/DDq_pIfHqLs

Conditional Score Function and Vector Field

$$\text{let } \mu_t(x_1 = z) = \alpha_t z, \text{ and } \sigma_t(x_1 = z) = \beta_t \quad \text{(where } \alpha_t \text{ and } \beta_t \text{ are differentiable functions of } t\text{)}$$
$$\text{where } \alpha_0 = \beta_1 = 0, \text{ and } \alpha_1 = \beta_0 = 1$$

For a Gaussian path

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t; \alpha_t z, \beta_t^2 \mathbf{I}_d)$$

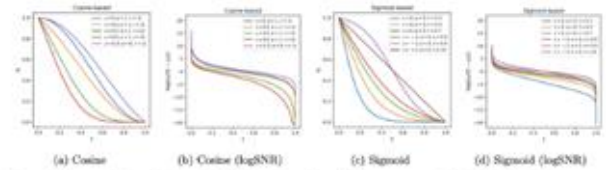


Figure 3: Instantiations of noise schedule function $\gamma(t)$ and the corresponding logSNR. Adjusting hyper-parameters of cosine and sigmoid functions leads to different noise schedules.

Recall: example of noise scheduling in diffusion

The conditional score function and the vector field are:

$$\nabla_{x_t} \log p_t(x_t|x_1 = z) = -\frac{x_t - \alpha_t z}{\beta_t^2} = \frac{\alpha_t}{\beta_t^2} z - \frac{1}{\beta_t^2} x_t$$
$$u_t(x_t|x_1 = z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t$$

These represent the interpolation between the original image z and the noisy image x_t at time t.

$$\boxed{\dot{x} = \frac{\mathrm{d}}{\mathrm{d}t} x}$$

proof in the appendix.

Conditional Score Function and Vector Field

$$\nabla_{x_t} \log p_t(x_t|x_1 = z) = -\frac{x_t - \alpha_t z}{\beta_t^2} \longrightarrow z = \frac{1}{\alpha_t}\left(x_t + \beta_t^2 \nabla \log p_t(x_t|z)\right)$$

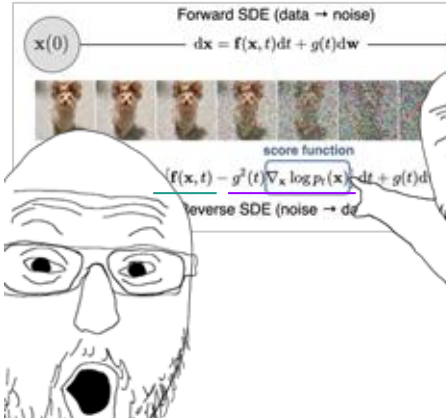
$$u_t(x_t|x_1 = z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t$$

plug z in to u_t

The result is:

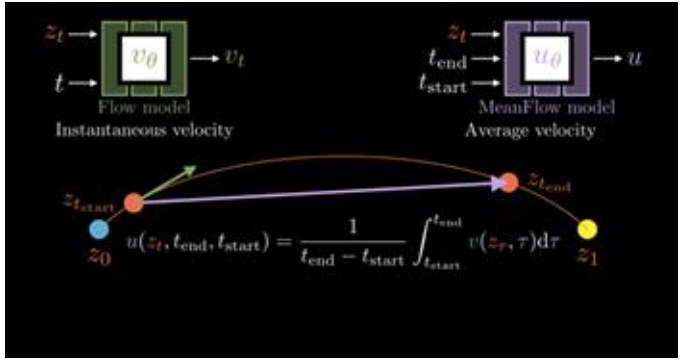
$$u_t(x_t|x_1 = z) = \underline{\frac{\dot{\alpha}_t}{\alpha_t} x_t} + \underline{\left(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t^2 - \dot{\beta}_t \beta_t\right)} \nabla \log p_t(x_t|z)$$

This will be applied later in the Classifier-Free Guidance in Flow Matching model.



Mean Flows

Traditional diffusion and flow-matching generative models generate high-quality images but require multiple, slow computational steps.



https://youtu.be/swKdn-qT47Q

Mean Flow models the average velocity of the flow field, rather than the instantaneous velocity.

Mean Flows for One-step Generative Modeling (Geng et al., 2025): https://arxiv.org/abs/2505.13447

Controlling the generation process

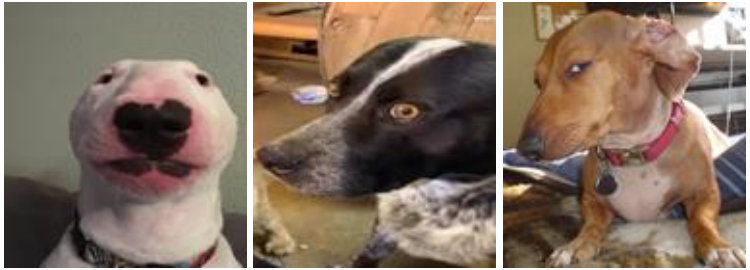
Controlling the generation process

The generative models we have studied so far utilize **unconditional generation**. This means the objective is to sample data x directly from the learned distribution p_{data} without any external guidance or control over the specific attributes of the output.

p_{data_1}

→ fixed label: y="cat"

p_{data_2}



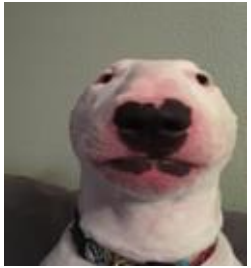
→ fixed label: y= "dog"

Conditional data distribution

Generating an image given a specific condition y (e.g., a text prompt or class label).

$$p_{\text{data}}(\cdot|y)$$

$p_{\text{data}}(\cdot|y = \text{"cat"})$



$p_{\text{data}}(\cdot|y = \text{"dog"})$

$p_{\text{data}}(\cdot|y = \text{"horse"})$

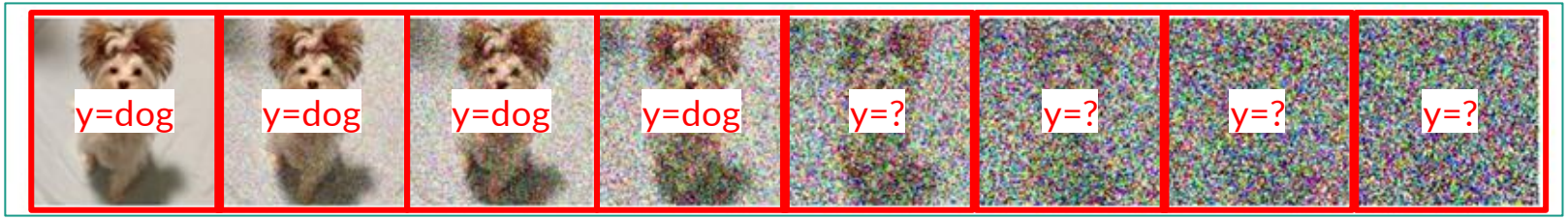
The objective is to model the conditional distribution p(x|y), allowing us to synthesize samples that satisfy specific constraints defined by the condition y.

Conditional score function (Diffusion)

To guide the diffusion model, we need a classifier model to acquire the updating direction, as shown in the following equation.

$$
\begin{aligned}
\nabla_x \log p_t(x|y) &= \nabla_x \log \frac{p_t(y|x)p_t(x)}{p_t(y)} \\
&= \nabla_x \log p_t(y|x) + \nabla_x \log p_t(x) + \cancel{\nabla_x \log p_t(y)}^{0} \\
&= \underbrace{\nabla_x \log p_t(x)}_{\text{score func}} + \underbrace{\nabla_x \log p_t(y|x)}_{\text{likelihood => classifier}}
\end{aligned}
$$

classifier input



However, we cannot directly employ the off-the-shelf model because the model must accommodate to predict the target given noisy image (x_t) for any t, which is a drawback of this approach.

Classifier Free Guidance (Diffusion)

In *Classifier-Free Diffusion Guidance 2022* paper, it propose to eliminate the classifier model from the following equation

Instead of relying on additional classifier, we can parameterize as an interpolation between predictors from diffusion model with and without labels.

guidance strength w > 1

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}|\mathbf{y}) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) + \underline{w} \nabla_{\mathbf{x}} \log p_t(\mathbf{y}|\mathbf{x})$$

$$\begin{aligned}
\epsilon(\mathbf{x}, \mathbf{y}) &= \epsilon(\mathbf{x}) - w \nabla_{\mathbf{x}} \log p_t(\mathbf{y}|\mathbf{x}) \\
&= \epsilon(\mathbf{x}) - w[\nabla_{\mathbf{x}} \log p_t(\mathbf{x}|\mathbf{y}) - \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] \\
&= \epsilon(\mathbf{x}) - w[-\epsilon(\mathbf{x}, \mathbf{y}) + \epsilon(\mathbf{x})] \\
&= (1 - w)\epsilon(\mathbf{x}) + w\epsilon(\mathbf{x}, \mathbf{y})
\end{aligned}$$

practical implementation

$$\boxed{\epsilon(\mathbf{x}) = \epsilon(\mathbf{x}, \mathbf{y} = \varnothing)}$$

We model these

https://arxiv.org/abs/2207.12598 **Classifier-Free Diffusion Guidance 2022**

Classifier Free Guidance (Diffusion) — Loss

$$\mathcal{L}_{\text{CSM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square}\left[\|\epsilon_\theta(x_t, y) - \epsilon\|^2\right]$$

image label/prompt

$$\square = (z, y) \sim p_{\text{data}}, t \sim \mathcal{U}[0, T], \epsilon \sim \mathcal{N}(0, \mathbf{I}), \text{replace } y = \varnothing \text{ with rate } \eta$$

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t|\alpha_t z, \beta_t^2 \mathbf{I}_d) \longrightarrow x_t = \alpha_t z + \beta_t \epsilon$$

Classifier Free Guidance (Diffusion)

Algorithm 1 Joint training a diffusion model with classifier-free guidance

Require: p_{uncond}: probability of unconditional training

repeat
- $(\mathbf{x}, \mathbf{c}) \sim p(\mathbf{x}, \mathbf{c})$ ▷ Sample data with conditioning from the dataset
- $\mathbf{c} \leftarrow \varnothing$ with probability p_{uncond} ▷ Randomly discard conditioning to train unconditionally
- $\lambda \sim p(\lambda)$ ▷ Sample log SNR value
- $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- $\mathbf{z}_\lambda = \alpha_\lambda \mathbf{x} + \sigma_\lambda \boldsymbol{\epsilon}$ ▷ Corrupt data to the sampled log SNR value
- Take gradient step on $\nabla_\theta \left\| \boldsymbol{\epsilon}_\theta(\mathbf{z}_\lambda, \mathbf{c}) - \boldsymbol{\epsilon} \right\|^2$ ▷ Optimization of denoising model

until converged

passing blank text “” to text encoder

Algorithm 2 Conditional sampling with classifier-free guidance

Require: w: guidance strength

Require: $\mathbf{c}$: conditioning information for conditional sampling

Require: $\lambda_1, \ldots, \lambda_T$: increasing log SNR sequence with $\lambda_1 = \lambda_{\min}$, $\lambda_T = \lambda_{\max}$

$\mathbf{z}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

for $t = 1, \ldots, T$ **do**

▷ Form the classifier-free guided score at log SNR λ_t

$\tilde{\boldsymbol{\epsilon}}_t = (1 + w)\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, \mathbf{c}) - w\boldsymbol{\epsilon}_\theta(\mathbf{z}_t)$

▷ Sampling step (could be replaced by another sampler, e.g. DDIM)

$\tilde{\mathbf{x}}_t = (\mathbf{z}_t - \sigma_{\lambda_t} \tilde{\boldsymbol{\epsilon}}_t) / \alpha_{\lambda_t}$

$\mathbf{z}_{t+1} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}_{\lambda_{t+1}|\lambda_t}(\mathbf{z}_t, \tilde{\mathbf{x}}_t), (\tilde{\sigma}^2_{\lambda_{t+1}|\lambda_t})^{1-v} (\sigma^2_{\lambda_t|\lambda_{t+1}})^v)$ if $t < T$ else $\mathbf{z}_{t+1} = \tilde{\mathbf{x}}_t$

end for

return $\mathbf{z}_{T+1}$

Classifier-Free Guidance for Flow Matching

$$
\begin{aligned}
u_t(x|y) &= a_t x + b_t \nabla \log p_t(x|y) \\
&= a_t x + b_t(\nabla \log p_t(x) + \nabla \log p_t(y|x)) \\
&= (a_t x + b_t \nabla \log p_t(x)) + b_t \nabla \log p_t(y|x) \\
&= u_t(x) + b_t \nabla \log \underline{p_t(y|x)}
\end{aligned}
$$

classifier

Add guidance weight w > 1:

$$\tilde{u}_t(x|y) = u_t(x) + w b_t \nabla \log p_t(y|x)$$

Rearranging the terms (see in the appendix):

$$\boxed{\tilde{u}_t(x|y) = (1-w)u_t(x) + w u_t(x|y)}$$

VF CFG

Conditional Score Function and Vector Field

$$\nabla_{x_t} \log p_t(x_t|x_1 = z) = -\frac{x_t - \alpha_t z}{\beta_t^2} = \frac{\alpha_t}{\beta_t^2} z - \frac{1}{\beta_t^2} x_t \longrightarrow z = \frac{1}{\alpha_t}(x_t + \beta_t^2 \nabla \log p_t(x_t|z))$$

$$u_t(x_t|x_1 = z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t$$

plug z into u_t

The result is:

$$u_t(x_t|x_1 = z) = \underbrace{\boxed{\frac{\dot{\alpha}_t}{\alpha_t}}}_{a_t} x_t + \underbrace{\boxed{\left(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t^2 - \dot{\beta}_t \beta_t\right)}}_{b_t} \nabla \log p_t(x_t|z)$$

(See Appendix for the full derivation)

This will be applied later in the Classifier-Free Guidance in Flow Matching model.

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t|\alpha_t z, \beta_t^2 \mathbf{I}_d)$$

$$u_t(x_t|x_1) = \sigma_t'(x_1)\frac{x_t - \mu_t(x_1)}{\sigma_t(x_1)} + \mu_t'(x_1)$$

Classifier Free Guidance for Flow Matching — Loss

$$\boxed{v_t^\theta(x|y) = (1-w)v_t^\theta(x) + wv_t^\theta(x|y)} \quad \boxed{v_t^\theta(x) = v_t^\theta(x|y=\varnothing)}$$

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square}\left[\left\|v_t^\theta(x_t|y) - u_t(x_t|z)\right\|^2\right]$$

image label/prompt

$$\square = (z, y) \sim p_{\text{data}}, t \sim \mathcal{U}[0,1], \epsilon \sim \mathcal{N}(0, \mathbf{I}), \text{replace } y = \varnothing \text{ with rate } \eta$$

$$p_t(x_t|x_1 = z) = \mathcal{N}(x_t|\alpha_t z, \beta_t^2 \mathbf{I}_d) \longrightarrow x_t = \alpha_t z + \beta_t \epsilon$$

Classifier Free Guidance for Flow Matching

Algorithm 5 Classifier-free guidance training for Gaussian probability path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

1: **for** each mini-batch of data **do**
2: Sample a data example (z, y) from the dataset.
3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
5: Set $x = \alpha_t z + \beta_t \epsilon$
6: With probability p drop label: $y \leftarrow \varnothing$
7: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x|y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2$$

8: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
9: **end for**

$$u_t(x_t|x_1 = z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t$$

$$x = \alpha_t z + \beta_t \epsilon$$

Algorithm 8 Classifier-Free Guidance Sampling Procedure

Require: A trained guided vector field $u_t^\theta(x|y)$.

1: Select a prompt $y \in \mathcal{Y}$, or take $y = \varnothing$ for unguided sampling.
2: Select a **guidance scale** $w > 1$.
3: Initialize $X_0 \sim p_{\text{init}}$.
4: Simulate $\mathrm{d}X_t = \left[(1-w)u_t^\theta(X_t|\varnothing) + w u_t^\theta(X_t|y)\right] \mathrm{d}t$ from $t = 0$ to $t = 1$.

Algorithm 1 Sampling from a Flow Model with Euler method

Require: Neural network vector field u_t^θ, number of steps n

1: Set $t = 0$
2: Set step size $h = \frac{1}{n}$
3: Draw a sample $X_0 \sim p_{\text{init}}$
4: **for** $i = 1, \ldots, n$ **do**
5: $X_{t+h} = X_t + h u_t^\theta(X_t)$
6: Update $t \leftarrow t + h$
7: **end for**
8: **return** X_1

Control Diffusion model through Text

Conditioning on text provides more flexibility than using a class label. Moreover, it can perform zero-shot learning.

DALL-E 2 proposed to be conditioned on CLIP embeddings. So, what is CLIP?

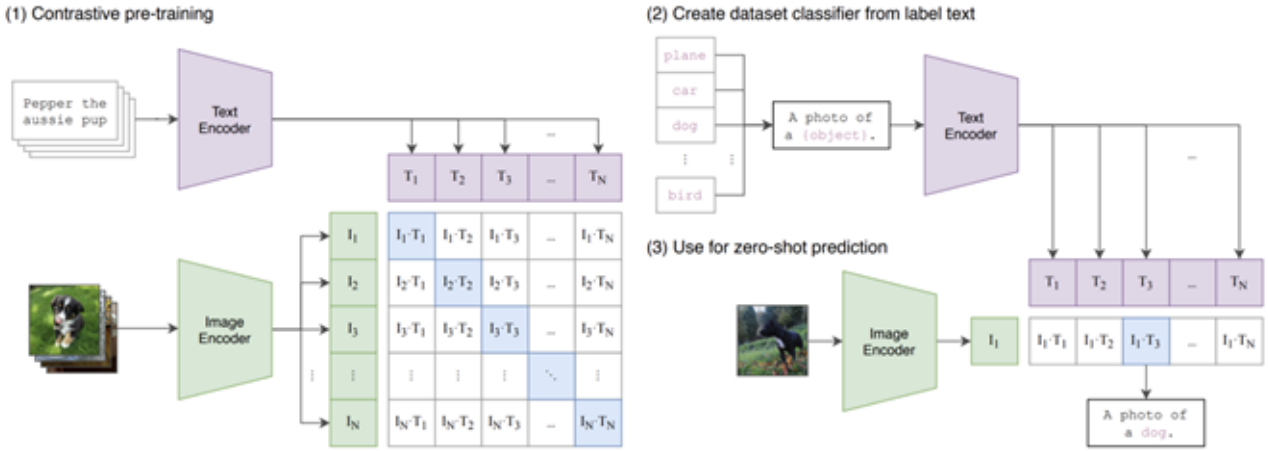
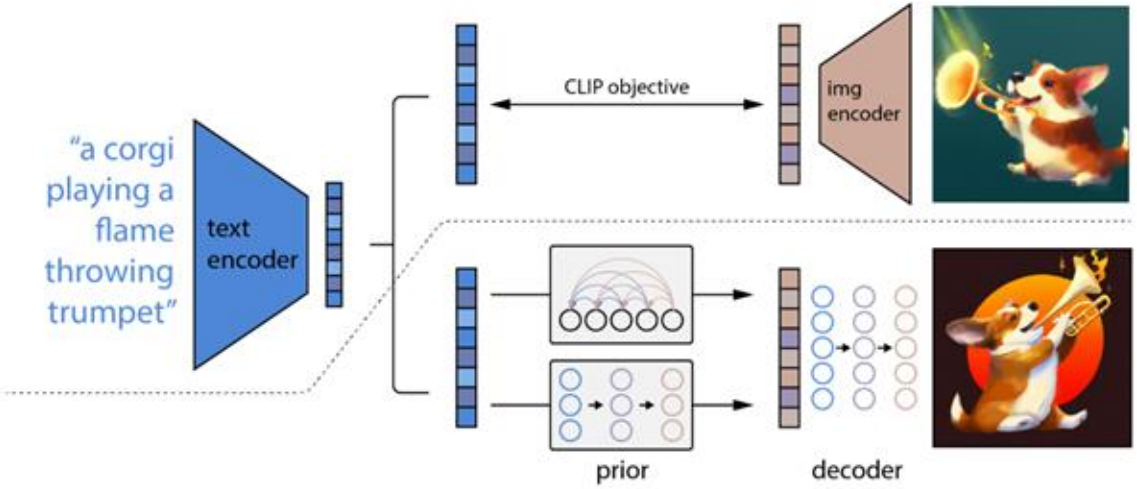


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

Control Diffusion model through Text

Step-by-step workflow of DALL-E 2

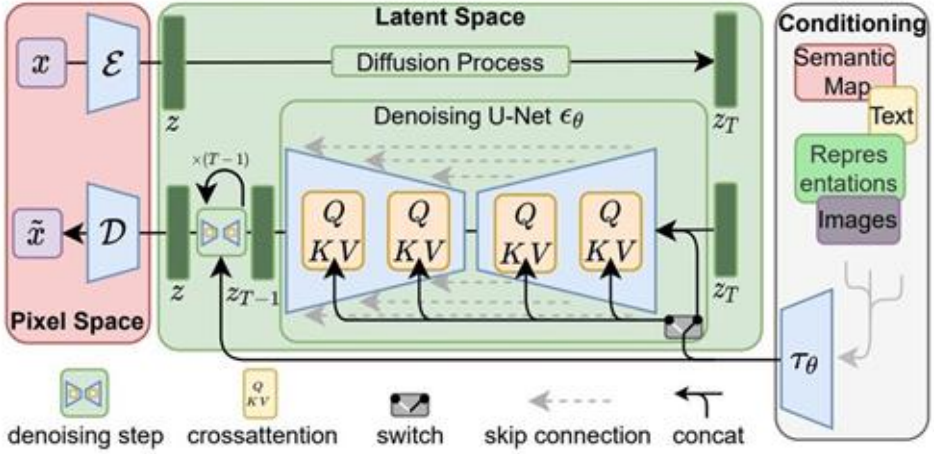
1. Text encoded using CLIP (CLIP Text Embeddings)
2. CLIP text embeddings converted to CLIP image embeddings via Prior (Autoregressive or Diffusion Model)
3. CLIP image embeddings to final image generation



Beyond Text Modality

Stable diffusion proposed cross-attention, which is effective for learning attention-based models for various input modalities. It turns diffusion models into powerful and flexible generators for general conditioning inputs.

Moreover, it operates in an embedding space, significantly reducing computational requirements

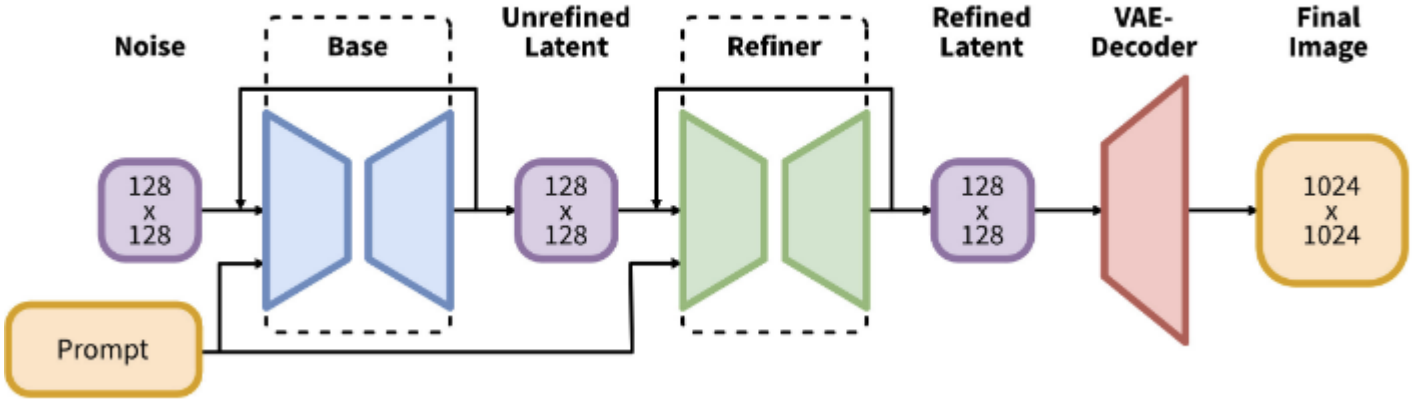


High-Resolution Image Synthesis with Latent Diffusion Models(2022): https://arxiv.org/abs/2112.10752

https://arxiv.org/abs/2307.01952

Stable Diffusion XL (SDXL)

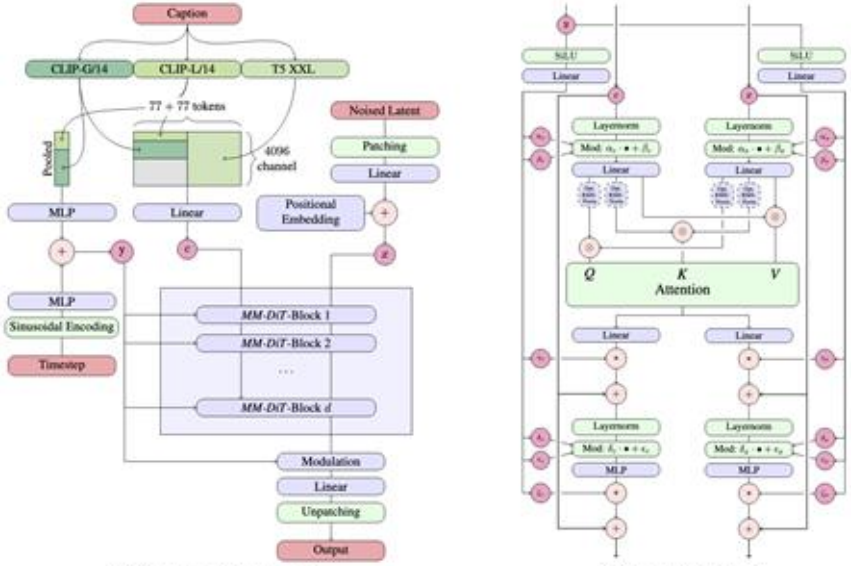
SDXL is built on the standard **Latent Diffusion Model (LDM)** framework. It generates images by *learning to reverse a specific forward process* that gradually adds noise to the data over time. Because standard diffusion relies on curved probability paths, the model requires many iterative integration steps to accurately simulate the reverse process and remove the noise.



https://arxiv.org/abs/2403.03206

Stable Diffusion 3

Stable Diffusion 3 introduces **Multimodal Diffusion Transformer (MM-DiT).**
Text and Image embeddings are conceptually quite different, this architecture uses separate sets of weights for the two modalities, allowing a bidirectional flow of information between image and text tokens within the network.

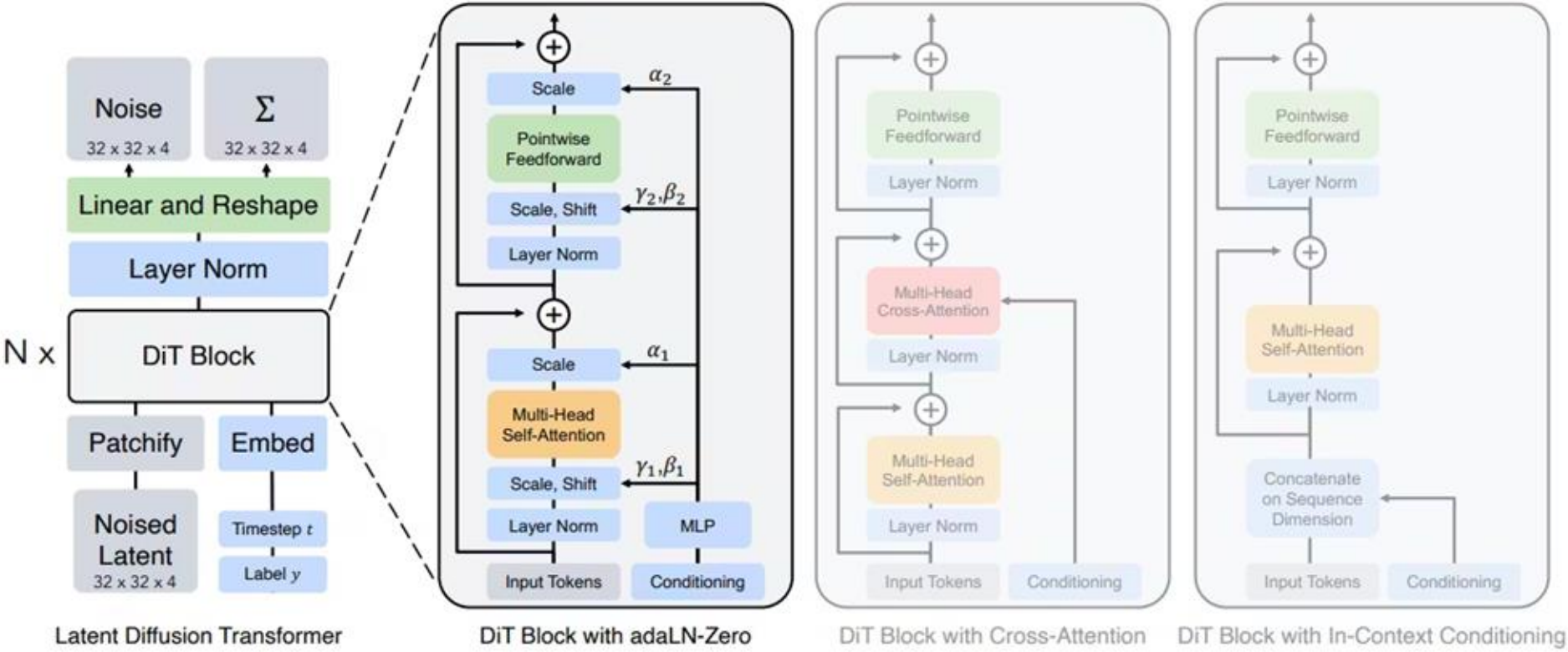


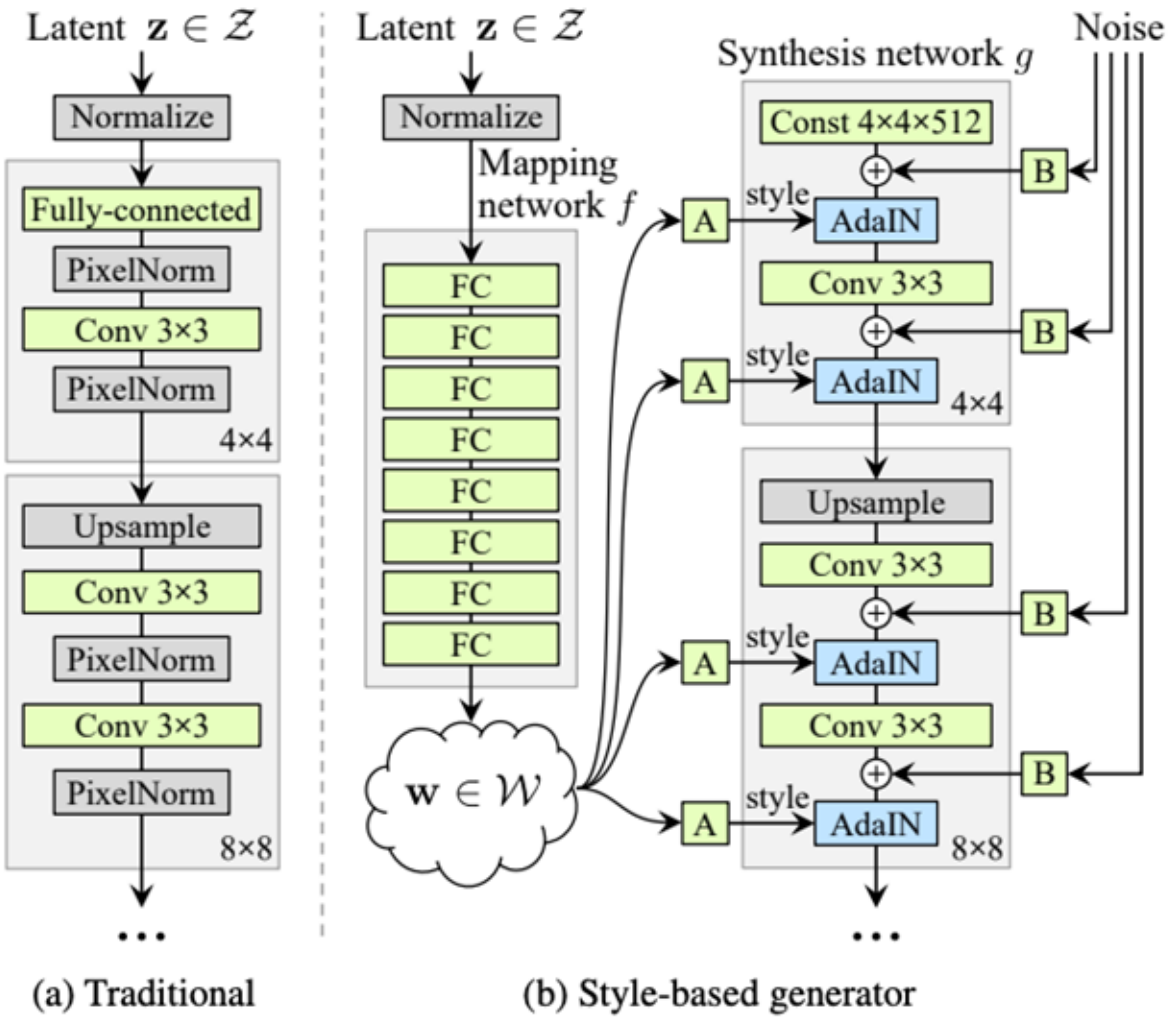
(a) Overview of all components.

(b) One *MM-DiT* block

Diffusion Transformer (DiT)

https://arxiv.org/abs/2212.09748





(a) Traditional

(b) Style-based generator

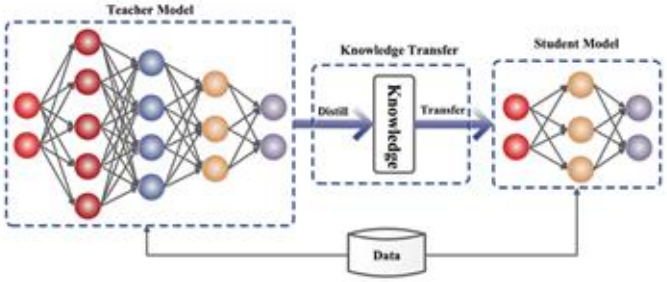
Adaptive Instance Norm
from StyleGANv1
is used to better condition the
generation

https://arxiv.org/abs/2506.15742

FLUX

FLUX & Latent Adversarial Diffusion Distillation (LADD)

- Standard Flow Matching generation requires high computationally expensive simulation steps.
- FLUX applies Latent Adversarial Diffusion Distillation (LADD) to compress this process. Using adversarial training, it teaches the model to output high-quality images in significantly fewer steps.



knowledge distillation
uses a larger well-trained teacher model to teach a smaller student model

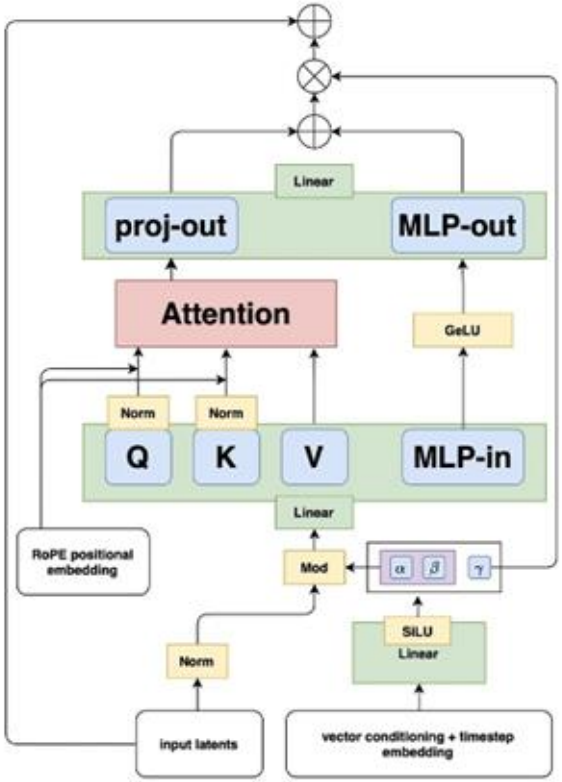


Figure 3: A fused DiT block equipped with rotary positional embeddings

LADD

https://arxiv.org/abs/2111.01007 shows that discriminating the in latent space converges faster.

Doing distillation in the latent space also removes the need for encoder and decoder computations

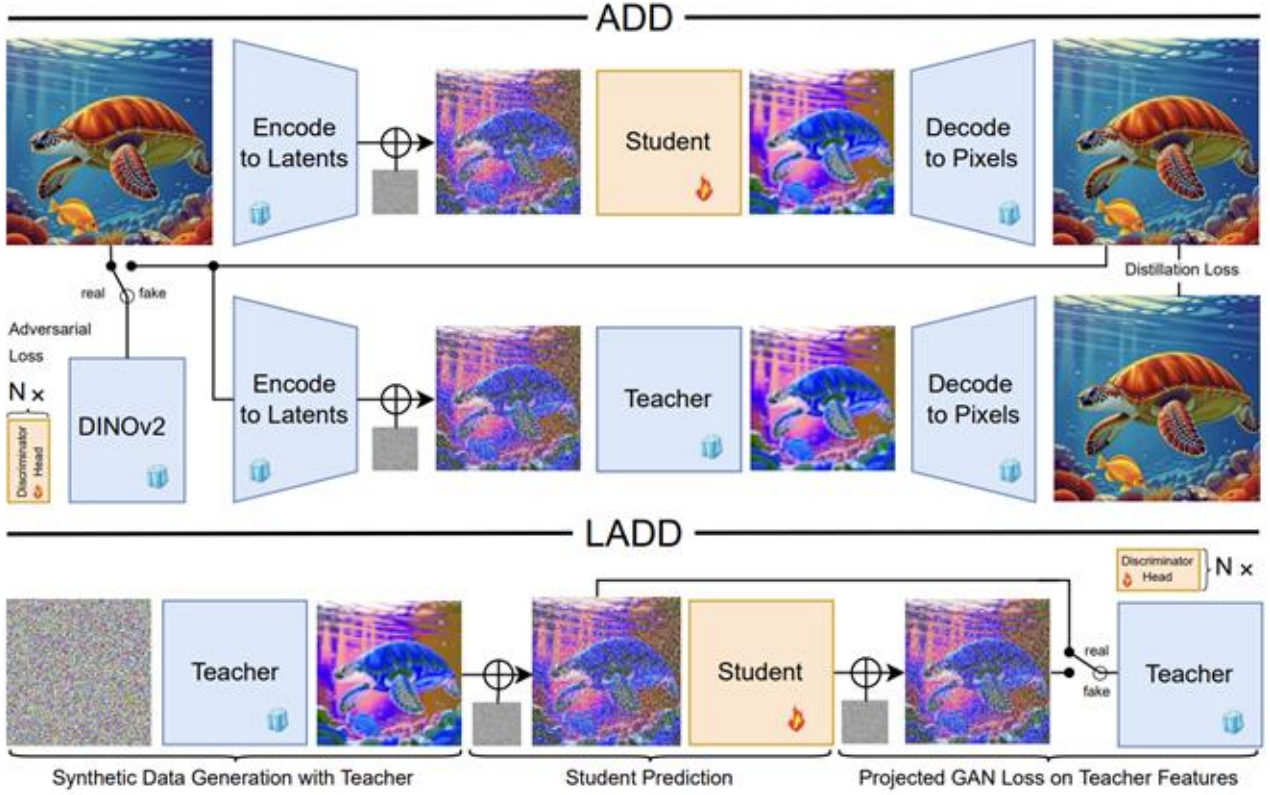


Figure 3: **Comparing ADD and LADD.** System overview and direct comparison to ADD. ADD (top two rows) computes a distillation loss in pixel space and an adversarial loss on top of DINOv2 features, thereby requiring expensive decoding from latent space to pixel space. In LADD (bottom row), we use the teacher model for synthetic data generation and its features for the adversarial loss, which allows us to train purely in the latent space. https://arxiv.org/abs/2403.12015

Parameter Efficient Fine-Tuning

PEFT

While scaling up deep learning models, several emerging properties occur, such as zero-shot or few-shot in LLM.

However, to finetune the model to a particular task is very cumbersome. Several researchers have been working on the model compressions:

1. Prefix tuning
   a. Append a learnable token in front of the input
2. Low-Rank Adaptation (LoRA)
   a. Represent an adaptation weight (gradient) with a low-rank matrix.
3. Adapter Module
   a. Insert a small number of layers that are relatively small compared to the entire model.

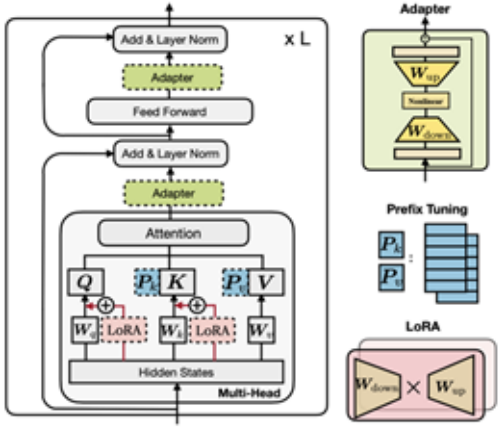
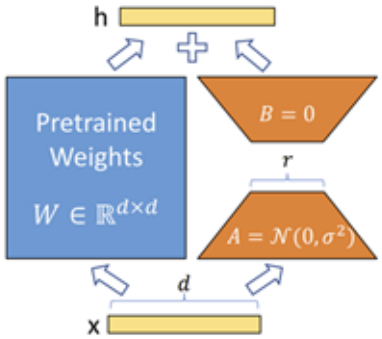
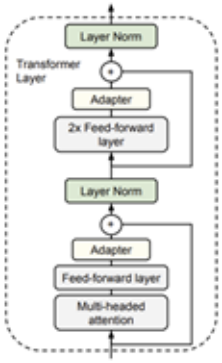
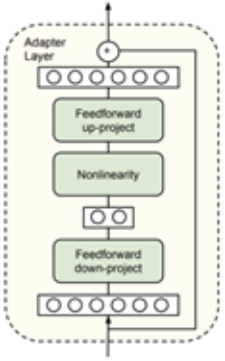


Figure 1: Illustration of the transformer architecture and several state-of-the-art parameter-efficient tuning methods. We use blocks with dashed borderlines to represent the added modules by those methods.







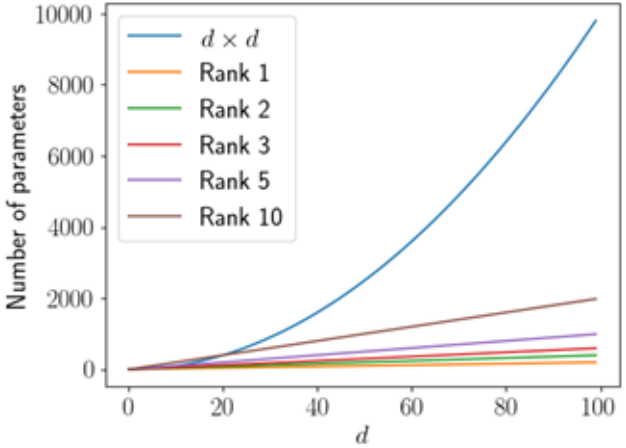
https://openreview.net/forum?id=0RDcd5Axok

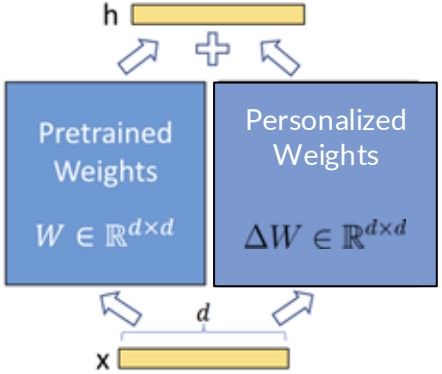
Low-Rank Adaptation (LoRA)

If we want to deploy a personalized large language model, we need to separately deploy each model for each user.

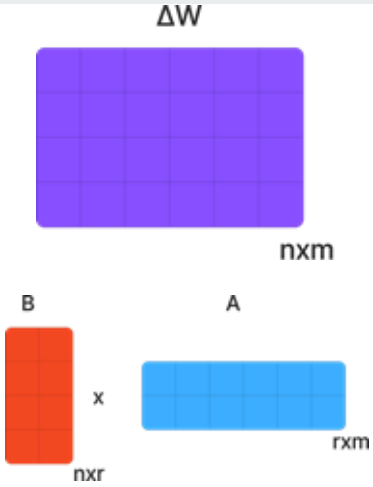
LoRA compress this personalized weights using low-rank decomposition

$$\Delta W = BA \qquad B \in \mathbb{R}^{d \times r} A \in \mathbb{R}^{r \times d}$$





Low-Rank Adaptation (LoRA)



In the initialization process, we use a random Gaussian initialization for A and zero for B.

$$h = W_0 x + \cancel{\Delta W} x = W_0 x + BAx$$

0

Therefore, the model is initialized to be identical to the pretrained weights.

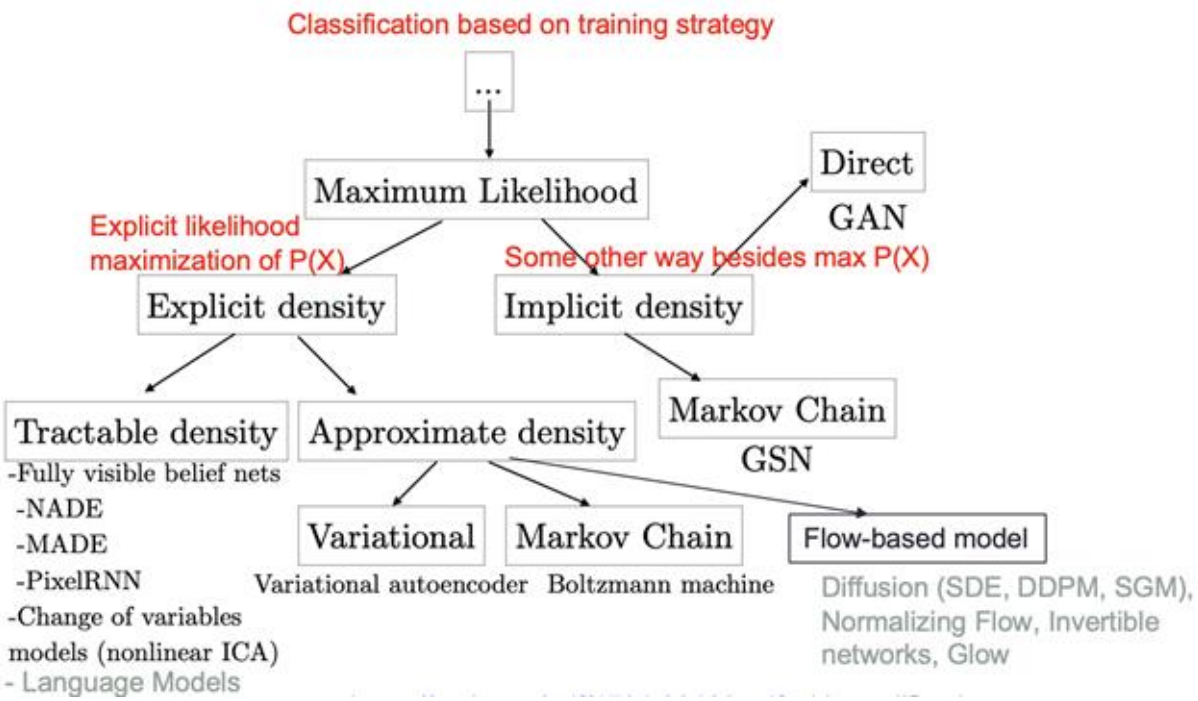
In addition, the paper proposes to scale low-rank matrices by alpha/r, claiming that it helps reduce the need to adjust hyperparameters when varying r.

Can be seen as Learning Rate

$$h = W_0 x + \frac{\boxed{\alpha}}{r} BAx$$

https://arxiv.org/abs/2106.09685 LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS 2021

Summary



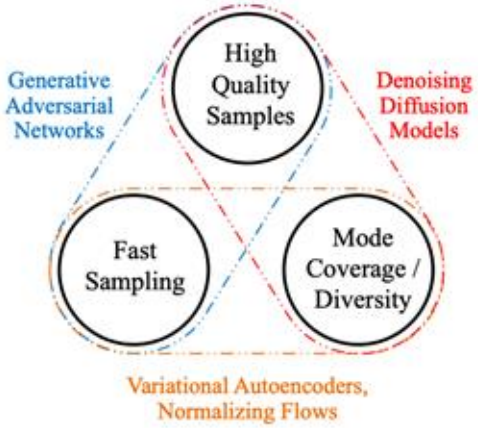


Figure 1: Generative learning trilemma.

Appendix

Condtional FM loss is equal to CFM loss plus a constant w.r.t. θ

$$
\begin{aligned}
\mathcal{L}_{\mathrm{FM}}(\theta) &= \mathbb{E}_{t,x_t}\left[\left\|v_t^\theta(x_t) - u_t(x_t)\right\|^2\right] \\
&= \mathbb{E}_{t,x_t}\left[\|v_t^\theta(x_t)\|^2 - 2\langle v_t^\theta(x_t), u_t(x_t)\rangle + \|u_t(x_t)\|^2\right] \\
&= \mathbb{E}_{t,x_t}\left[\|v_t^\theta(x_t)\|^2\right] - 2\mathbb{E}_t\left[\int \langle v_t^\theta(x_t), u_t(x_t)\rangle p_t(x_t)dx_t\right] + \mathbb{E}_{t,x_t}\left[\|u_t(x_t)\|^2\right] \\
&= \mathbb{E}_{t,x_t}\left[\|v_t^\theta(x_t)\|^2\right] - 2\mathbb{E}_t\left[\int \left\langle v_t^\theta(x_t), \int u_t(x_t|z)p_t(x_t|z)p(z)dz\right\rangle dx_t\right] + \mathbb{E}_{t,x_t}\left[\|u_t(x_t)\|^2\right] \\
&= \mathbb{E}_{t,z,x_t}\left[\|v_t^\theta(x_t)\|^2\right] - 2\mathbb{E}_{t,z,x_t}\left[\langle v_t^\theta(x_t), u_t(x_t|z)\rangle\right] + \mathbb{E}_{t,x_t}\left[\|u_t(x_t)\|^2\right] \\
&= \mathbb{E}_{t,z,x_t}\left[\|v_t^\theta(x_t)\|^2 - 2\langle v_t^\theta(x_t), u_t(x_t|z)\rangle + \|u_t(x_t|z)\|^2\right] - \mathbb{E}_{t,z,x_t}\left[\|u_t(x_t|z)\|^2\right] + \mathbb{E}_{t,x_t}\left[\|u_t(x_t)\|^2\right] \\
&= \mathbb{E}_{t,z,x_t}\left[\left\|v_t^\theta(x_t) - u_t(x_t|z)\right\|^2\right] + \underbrace{\mathbb{E}_{t,x_t}\left[\|u_t(x_t)\|^2\right] - \mathbb{E}_{t,z,x_t}\left[\|u_t(x_t|z)\|^2\right]}_{\text{Constant } C \text{ (independent of } \theta)} \\
&= \mathcal{L}_{\mathrm{CFM}}(\theta) + C
\end{aligned}
$$

Appendix

conditional score function for α_t and β_t

$$
\begin{aligned}
p_t(x_t|x_1 = z) &= \mathcal{N}(x_t; \alpha_t z, \beta_t^2 \mathbf{I}_d) \\
&= \frac{1}{\sqrt{(2\pi)^d |\beta_t^2 \mathbf{I}_d|}} \exp\left(-\frac{1}{2}(x_t - \alpha_t z)^T (\beta_t^2 \mathbf{I}_d)^{-1} (x_t - \alpha_t z)\right) \\
&= C \exp\left(-\frac{1}{2\beta_t^2}(x_t - \alpha_t z)^T (x_t - \alpha_t z)\right) \\
\log p_t(x_t|x_1 = z) &= \log(C) - \frac{1}{2\beta_t^2}(x_t - \alpha_t z)^T (x_t - \alpha_t z) \\
\nabla_{x_t} \log p_t(x_t|x_1 = z) &= \nabla_{x_t}\left(\log(C) - \frac{1}{2\beta_t^2}(x_t - \alpha_t z)^T (x_t - \alpha_t z)\right) \\
&= -\frac{1}{2\beta_t^2} \nabla_{x_t}\left((x_t - \alpha_t z)^T (x_t - \alpha_t z)\right) \\
&= -\frac{1}{2\beta_t^2} \cdot 2(x_t - \alpha_t z) \\
&= -\frac{x_t - \alpha_t z}{\beta_t^2}
\end{aligned}
$$

Appendix

conditional vector field for α_t and β_t

$$
\begin{aligned}
u_t(x_t|x_1 = z) &= \sigma_t'(x_1 = z)\frac{x_t - \mu_t(x_1 = z)}{\sigma_t(x_1 = z)} + \mu_t'(x_1 = z) \\
&= \dot{\beta}_t \frac{x_t - \alpha_t z}{\beta_t} + \dot{\alpha}_t z \\
&= \frac{\dot{\beta}_t}{\beta_t} x_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t z + \dot{\alpha}_t z \\
&= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t
\end{aligned}
$$

Appendix

Solve the relation between conditional score function and vector field

$$
\begin{aligned}
u_t(x_t|x_1 = z) &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x_t \\
&= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) \left[\frac{1}{\alpha_t}(x_t + \beta_t^2 \nabla \log p_t(x_t|z))\right] + \frac{\dot{\beta}_t}{\beta_t} x_t \\
&= \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)(x_t + \beta_t^2 \nabla \log p_t(x_t|z)) + \frac{\dot{\beta}_t}{\beta_t} x_t \\
&= \frac{\dot{\alpha}_t}{\alpha_t} x_t - \frac{\dot{\beta}_t}{\beta_t} x_t + \beta_t^2 \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right) \nabla \log p_t(x_t|z) + \frac{\dot{\beta}_t}{\beta_t} x_t \\
&= \frac{\dot{\alpha}_t}{\alpha_t} x_t + \beta_t^2 \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right) \nabla \log p_t(x_t|z) \\
&= \frac{\dot{\alpha}_t}{\alpha_t} x_t + \left(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t^2 - \dot{\beta}_t \beta_t\right) \nabla \log p_t(x_t|z)
\end{aligned}
$$

Appendix

Classifier free guidance for Vector Field

$$u_t(x|y) = u_t(x) + b_t \nabla \log p_t(y|x)$$
$$b_t \nabla \log p_t(y|x) = u_t(x|y) - u_t(x)$$

$$\begin{aligned}
\tilde{u}_t(x|y) &= u_t(x) + w\underline{b_t \nabla \log p_t(y|x)} \\
&= u_t(x) + w\Big(u_t(x|y) - u_t(x)\Big) \\
&= u_t(x) + wu_t(x|y) - wu_t(x) \\
&= (1-w)u_t(x) + wu_t(x|y)
\end{aligned}$$